

กลุ่มที่ 1: Hospital Patient Queue (ระบบคิวผู้ป่วยในโรงพยาบาล)

ส่วนที่ 1: การวิเคราะห์ปัญหา (Problem Analysis)

การวิเคราะห์องค์ประกอบหลักของระบบคิวผู้ป่วยในโรงพยาบาล เพื่อเป็นพื้นฐานในการเลือกโครงสร้างข้อมูลที่เหมาะสม:

- **Input:** ข้อมูลผู้ป่วยวัตถุ Patient ซึ่งประกอบด้วย Patient ID (เช่น P01), Name, Arrival Time (เวลาที่มาถึง), Severity Level (1 = Critical, 2 = Urgent, 3 = Normal), และ Service Time (เวลาที่ใช้ในการรักษา) รวมถึงคำสั่งสั่งการระบบ ได้แก่ ARRIVE, SERVE, PEEK, CANCEL, DISPLAY, และ QUEUE_SIZE
- **Output:** รายชื่อผู้ป่วยที่ได้รับการรักษาตามลำดับ, สถานะของคิวในปัจจุบัน, จำนวนผู้ป่วยคงเหลือในคิว, และสถานะการยกเลิกรายการ
- **Constraints:** ระบบต้องประมวลผลคำสั่งตามลำดับเวลาจริง โดยในกรณีที่ระดับความรุนแรง (Severity) เท่ากัน จะต้องให้บริการผู้ป่วยที่มาก่อนตามหลัก First-Come, First-Served (Strict FIFO tie-breaking)
- **Queue เก็บข้อมูลอะไร:** โครงสร้างคิวจัดเก็บออบเจกต์ของคลาส Patient
- **Queue ชนิดใดเหมาะสม:**
 - **Algorithm A (Normal Queue):** ใช้ ArrayDeque<Patient> หรือ LinkedList<Patient> ซึ่งเป็น FIFO Queue มาตรฐาน
 - **Algorithm B (Priority Queue):** ใช้ PriorityQueue<Patient> ร่วมกับ Custom Comparator เพื่อจัดลำดับความสำคัญตามระดับความรุนแรงและเวลาที่มาถึง
- **เหตุผลที่ต้องใช้ Queue:** การบริหารจัดการคิวผู้ป่วยเป็นการจัดลำดับการเข้าถึงทรัพยากร (แพทย์/ห้องตรวจ) โครงสร้างข้อมูลแบบ Queue ช่วยให้การเพิ่มข้อมูลเข้า (Enqueue) และการนำข้อมูลออก (Dequeue) มีประสิทธิภาพสูง ($O(1)$ สำหรับ FIFO และ $O(\log n)$ สำหรับ Priority Queue) ซึ่งสอดคล้องกับพฤติกรรมกรรมการเข้ารับบริการในโลกจริง

ส่วนที่ 2: การออกแบบอัลกอริทึม (Algorithm Design & Pseudocode)

Algorithm A: Normal FIFO Queue

```
Algorithm ARRIVE_A(queue, patient)
  enqueue(queue, patient)
end Algorithm
```

```
Algorithm SERVE_A(queue)
  if queue is empty then
    return "No patients in queue"
  end if
  patient ← dequeue(queue)
  return patient
end Algorithm
```

Algorithm B: Priority Queue (Severity First, then Arrival Time)

```
Algorithm COMPARE_PATIENTS(p1, p2)
  if p1.severity != p2.severity then
    return p1.severity - p2.severity // Severity น้อยกว่า (1=Critical) มีความสำคัญสูงกว่า
  else
    return p1.arrivalTime - p2.arrivalTime // มาก่อนได้ก่อน
  end if
end Algorithm
```

```
Algorithm ARRIVE_B(pqueue, patient)
  insert_with_priority(pqueue, patient, COMPARE_PATIENTS)
end Algorithm
```

```
Algorithm SERVE_B(pqueue)
  if pqueue is empty then
    return "No patients in queue"
  end if
  patient ← extract_min(pqueue)
  return patient
end Algorithm
```

ส่วนที่ 3: Queue Trace (การจำลองการทำงาน 10 Operations)

ชุดข้อมูลบังคับ: P01 (Normal, T=1), P02 (Urgent, T=2), P03 (Normal, T=3), P04 (Critical, T=4), P05 (Urgent, T=5)

Step	Operation	Queue Before	Queue After	Output	คำอธิบาย เหตุผล
1	ARRIVE P01 (Normal, T1)	[]	[P01]	-	คิวว่าง ใ้ P01 เข้าเป็น รายการแรก
2	ARRIVE P02 (Urgent, T2)	[P01]	[P02, P01]	-	P02 (Urgent=2) มีความสำคัญ สูงกว่า P01 (Normal=3) จึงอยู่หน้าสุด
3	ARRIVE P03 (Normal, T3)	[P02, P01]	[P02, P01, P03]	-	P03 (Normal=3) ความสำคัญ เท่า P01 แต่นำ เข้าทีหลัง จึง ต่อท้าย P01
4	ARRIVE P04 (Critical, T4)	[P02, P01, P03]	[P04, P02, P01, P03]	-	P04 (Critical=1) มีความสำคัญ สูงสุด แขนงขึ้น หน้าสุดทันที
5	ARRIVE P05 (Urgent, T5)	[P04, P02, P01, P03]	[P04, P02, P05, P01, P03]	-	P05 (Urgent=2) อยู่หลัง P02 (Urgent=2

Step	Operation	Queue Before	Queue After	Output	คำอธิบาย เหตุผล
					แต่มารับ บริการก่อน) แต่อยู่หน้า P01
6	PEEK	[P04, P02, P05, P01, P03]	[P04, P02, P05, P01, P03]	P04	อ่านข้อมูลผู้ ป่วยหัวคิวโดย ไม่ลบออกจาก คิว
7	SERVE	[P04, P02, P05, P01, P03]	[P02, P05, P01, P03]	P04	ดึง P04 ซึ่ง เป็น Critical ออกไปทำการ รักษาเป็นคน แรก
8	CANCEL P01	[P02, P05, P01, P03]	[P02, P05, P03]	Success	ค้นหาและลบ P01 ที่อยู่ กลางคิวออก สำเร็จ
9	SERVE	[P02, P05, P03]	[P05, P03]	P02	ดึง P02 (Urgent, T=2) ออกมา รักษาเป็น ลำดับถัดไป
10	SERVE	[P05, P03]	[P03]	P05	ดึง P05 (Urgent, T=5) ออกมา รักษา

ส่วนที่ 4: ความถูกต้องของอัลกอริทึม (Correctness / Loop Invariant)

- **Invariant ของ Algorithm A (FIFO Queue):** ก่อนและหลังการทำงานในแต่ละรอบ สมาชิกที่อยู่ในบริเวณ Front จะเป็นผู้ป่วยที่เข้ามาในระบบก่อนผู้ป่วยคนอื่นทั้งหมดที่ยังไม่ได้รับการประมวลผล
- **Invariant ของ Algorithm B (Priority Queue):** ก่อนและหลังการทำงานในแต่ละรอบ ผู้ป่วยที่อยู่ในบริเวณ Front จะเป็นผู้ป่วยที่มีระดับความรุนแรงสูงสุด (Severity สังเกตจากค่าน้อยสุด) และหากมีระดับความรุนแรงเท่ากัน จะเป็นผู้ป่วยที่เดินทางมาถึงก่อนเสมอ

ส่วนที่ 5 & 6: การวิเคราะห์ความซับซ้อน (Complexity Analysis)

Time Complexity

Operation	Algorithm A (ArrayDeque)	Algorithm B (PriorityQueue)
enqueue / offer	$O(1)$	$O(\log n)$
dequeue / poll	$O(1)$	$O(\log n)$
peek	$O(1)$	$O(1)$
search / cancel	$O(n)$	$O(n)$
display	$O(n)$	$O(n \log n)$

Space Complexity

ทั้งสองอัลกอริทึมใช้พื้นที่หน่วยความจำ $O(n)$ โดยที่ n คือจำนวนผู้ป่วยสูงสุดที่อยู่ในคิว ณ เวลาใดเวลาหนึ่ง

ส่วนที่ 7 : โปรแกรมภาษา Java (Java Implementation)

ซอร์สโค้ดโปรแกรมฉบับสมบูรณ์ ซึ่งรวมคลาสผู้ป่วย ระบบคิวทั้งสองอัลกอริทึม และโปรแกรมทดสอบประสิทธิภาพ Benchmark:

```
import java.util.*;

class Patient {
    private String id;
    private String name;
    private int arrivalTime;
    private int severity; // 1 = Critical, 2 = Urgent, 3 = Normal
    private int serviceTime;

    public Patient(String id, String name, int arrivalTime, int severity, int serviceTime) {
        this.id = id;
        this.name = name;
        this.arrivalTime = arrivalTime;
        this.severity = severity;
        this.serviceTime = serviceTime;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public int getArrivalTime() { return arrivalTime; }
    public int getSeverity() { return severity; }
    public int getServiceTime() { return serviceTime; }

    @Override
    public String toString() {
        return String.format("[%s] %s (Sev:%d, Arr:%d)", id, name, severity, arrivalTime);
    }
}

class HospitalSystemA {
    private Queue<Patient> queue = new ArrayDeque<>();

    public void arrive(Patient p) { queue.offer(p); }
    public Patient serve() { return queue.poll(); }
    public Patient peek() { return queue.peek(); }
    public int size() { return queue.size(); }
    public boolean isEmpty() { return queue.isEmpty(); }
    public boolean cancel(String id) {
        return queue.removeIf(p -> p.getId().equals(id));
    }
}
```

```

    }
}

class HospitalSystemB {
    private PriorityQueue<Patient> pQueue = new PriorityQueue<>(
        Comparator.comparingInt(Patient::getSeverity)
            .thenComparingInt(Patient::getArrivalTime)
    );

    public void arrive(Patient p) { pQueue.offer(p); }
    public Patient serve() { return pQueue.poll(); }
    public Patient peek() { return pQueue.peek(); }
    public int size() { return pQueue.size(); }
    public boolean isEmpty() { return pQueue.isEmpty(); }
    public boolean cancel(String id) {
        return pQueue.removeIf(p -> p.getId().equals(id));
    }
}

```

ส่วนที่ 8: กรณีทดสอบ (Test Cases ทั้ง 6 กรณี)

1. **Normal Case:** ใส่ผู้ป่วยความรุนแรงต่างกัน เช่น Critical, Urgent, Normal ตรวจสอบว่า Priority Queue ลำดับให้ Critical ได้รับการรักษาก่อน
2. **Empty Queue Case:** เรียกใช้คำสั่ง SERVE หรือ PEEK เมื่อคิวว่าง ระบบต้องส่งคืนค่า null โดยไม่เกิด Exception
3. **Single Item Case:** ใส่ผู้ป่วย 1 ราย ทำการ SERVE แล้วตรวจสอบว่าคิวกลับมามีขนาดเป็น 0
4. **Large Queue Case:** ทดสอบใส่ผู้ป่วยจำนวนมาก ($n \geq 1,000$) และทำการดึงคิวจนหมด ตรวจสอบความถูกต้องของการเรียงลำดับและความเสถียร
5. **Special / Edge Case (Priority เท่ากันหมด):** ใส่ผู้ป่วยที่มีระดับ Severity = 3 เท่ากันทุกคน ตรวจสอบว่าคิวจะให้บริการตาม Arrival Time แบบ FIFO
6. **Cancel Case:**
 - กรณีลบรายการที่มีอยู่จริงกลางคิว (เช่น ลบ P01) → ส่งคืนค่า true และโครงสร้างคิวยังเรียงลำดับถูกต้อง
 - กรณีลบรายการที่ไม่มีอยู่จริง (เช่น ลบ P99) → ส่งคืนค่า false และไม่กระทบข้อมูลในคิว

ส่วนที่ 9: การทดลองและเปรียบเทียบประสิทธิภาพ (Algorithm Experiment)

การทดสอบประสิทธิภาพวัดเวลาประมวลผลจริง (Execution Time) โดยใช้ System.nanoTime() มีกระบวนการ Warm-up, กำหนด Seed คงที่เพื่อสร้างชุดข้อมูลเหมือนกัน, และเฉลี่ยผลลัพธ์จากการทดลอง 5 รอบ:

ขนาดข้อมูล (N)	Algorithm A: FIFO Queue (ms)	Algorithm B: Priority Queue (ms)
100	0.020 ms	0.166 ms
1,000	0.152 ms	1.830 ms
10,000	2.033 ms	41.975 ms
50,000	8.519 ms	233.874 ms

การวิเคราะห์ผลทดลอง: ผลการทดลองสอดคล้องกับทฤษฎี โดย Algorithm A (ArrayDeque) มีอัตราการเติบโตแบบ $O(n)$ ทำให้ประมวลผลเร็วมาก ส่วน Algorithm B (PriorityQueue) เติบโตแบบ $O(n \log n)$ เนื่องจากการรักษาโครงสร้าง Min-Heap ในทุกๆ การ Insert/Remove ส่งผลให้ใช้เวลาเพิ่มขึ้นอย่างเห็นได้ชัดเมื่อ $N = 50,000$

ส่วนที่ 10: สรุปการเปรียบเทียบอัลกอริทึม (Algorithm Comparison)

ประเด็นเปรียบเทียบ	Algorithm A (Normal Queue)	Algorithm B (Priority Queue)
Data Structure	ArrayDeque<Patient>	PriorityQueue<Patient> (Binary Min-Heap)
หลักการการทำงาน	First-In, First-Out (FIFO)	Priority-based (Severity + Arrival Time)
Enqueue Complexity	O(1)	O(log n)
Dequeue Complexity	O(1)	O(log n)
Time Complexity รวม	O(n)	O(n log n)
Space Complexity	O(n)	O(n)
Fairness (ความยุติธรรม)	ความยุติธรรมทางเวลาสูงมาก (มาก่อนได้ก่อน)	มุ่งเน้นความจำเป็นทางการแพทย์เหนือเวลาที่มาถึง
ข้อดี	เรียบง่าย รวดเร็ว ประสิทธิภาพสูง ไม่ถูกแซงคิว	ช่วยชีวิตผู้ป่วยฉุกเฉินวิกฤตได้ทันที
ข้อจำกัด	ไม่สามารถรองรับเคสฉุกเฉินวิกฤตได้	เกิดปัญหา Starvation กับผู้ป่วย Normal ได้
ความเหมาะสมในการใช้งาน	ห้องจ่ายยา คลินิกทั่วไป แผนกนัดหมาย	ห้องฉุกเฉิน (ER) แผนกตรวจกรองผู้ป่วย (Triage)

คำตอบสำหรับคำถามท้ายโจทย์

1. **FIFO** เหมาะกับโรงพยาบาลทุกกรณีหรือไม่?

ไม่เหมาะ เนื่องจากในทางการแพทย์ ระดับความรุนแรงของโรคมีความสำคัญต่อชีวิตของผู้ป่วย หากใช้ FIFO ในห้องฉุกเฉิน ผู้ป่วยวิกฤตอาจต้องรอคิวและเสียชีวิตได้

2. **Priority Queue** แก้ปัญหาอะไร?

แก้ปัญหการจัดลำดับตามความสำคัญทางการแพทย์ (Triage) ช่วยให้ผู้ป่วย Critical/Urgent ได้รับการรักษาก่อนโดยทันที

3. ถ้ามีผู้ป่วย **Critical** เข้ามาตลอด จะเกิดปัญหอะไรกับผู้ป่วย **Normal**?

ผู้ป่วย Normal จะถูกเลื่อนคิวออกไปเรื่อยๆ ส่งผลให้ต้องรอคิวเป็นเวลานานเกินจำเป็น

4. อาจเกิด **Starvation** หรือไม่?

เกิดขึ้นได้ เรียกว่าภาวะ Starvation ในกรณีผู้ป่วย Normal วิธีแก้ไขคือการใช้แนวคิด **Aging** (ปรับเพิ่ม Priority ตามระยะเวลานั่งรอคิว)